

DATA ANALYTICS · WEEK 4 SUBMISSION

Statistics, Data Visualization & Exploratory Data Analysis

Retail Sales Analysis using Python

Dataset	retail_sales.csv (82 orders · 4 regions · 8 months)
Tools	Python, Pandas, NumPy, Matplotlib, Seaborn
Scope	Central tendency, dispersion, correlation, probability, outlier detection, 9 visualizations, data cleaning, insights & recommendations

Project Aim

To apply statistical techniques, data visualization methods and Exploratory Data Analysis (EDA) using Python on a retail sales dataset. The project calculates measures of central tendency and dispersion, studies correlation and probability, detects outliers, cleans and validates the data, builds nine meaningful Matplotlib and Seaborn visualizations, extracts data-driven insights, and translates those insights into actionable business recommendations.

Dataset Description

The dataset records 82 retail orders placed across four regions (North, South, East, West) over an eight-month window, spanning the Electronics, Furniture and Stationery categories. Each row captures an Order ID, order date, region, category, product, sales value, quantity, discount rate, profit and a customer satisfaction rating. One duplicate record and one deliberately high-value transaction were included to give the cleaning and outlier-detection steps something real to catch.

Report Contents

Task 1	Mean, Median and Mode
Task 2	Variance and Standard Deviation
Task 3	Correlation and Probability
Task 4	Outlier Detection using IQR
Task 5	Matplotlib Visualizations (5 charts)
Task 6	Seaborn Visualizations (4 charts)
Task 7	EDA - Data Inspection & Cleaning
Task 8	Correlation & Key Insights
Task 9	Business Recommendations

Task 1: Mean, Median and Mode

Aim:

Compute the three core measures of central tendency for the Sales column to understand the typical order value.

DESCRIPTION:

Central tendency summarizes a whole distribution with a single representative number, and the three classic measures rarely agree when a dataset is skewed. Here the mean pulls far above the median because a handful of very large Electronics orders inflate the average even though most transactions are modest. The median is the more trustworthy "typical" value in this case, since it ignores the pull of extreme rows. The mode reveals which specific price points repeat, which happens when the same product is sold more than once at an identical price. Reporting all three together, rather than any single statistic in isolation, gives a much fuller picture of how sales values are actually distributed. This combination is the standard first step before deciding whether later statistical tests need to account for skew. It also foreshadows the outlier investigation carried out in Task 4.

Source Code:

```
sales = df["Sales"]

mean_sales = sales.mean()
median_sales = sales.median()
mode_sales = sales.mode()

print("Mean Sales   :", round(mean_sales, 2))
print("Median Sales :", round(median_sales, 2))
print("Mode Sales   :", mode_sales.tolist())
```

Output:

```
Mean Sales   : 38517.56
Median Sales : 24398.4
Mode Sales   : [587.1, 638.4, 689.7, 805.6, 874.0, 942.4, 86982.0, 94164.0, 250000.0]
```

Interpretation: The mean (₹38,517.56) sits well above the median (₹24,398.40), a classic sign of right-skew driven by a small number of very high-value Electronics orders. The nine modal values indicate several exact price points were each sold more than once across the dataset.

Task 2: Variance and Standard Deviation

Aim:

Measure how far individual sales values spread out around the mean, quantifying overall volatility in order value.

DESCRIPTION:

While central tendency answers "what is typical," dispersion answers "how much do values vary." Variance squares each deviation from the mean, which weights larger deviations far more heavily than small ones, and standard deviation brings that number back into the original Sales units so it can be read intuitively. A standard deviation of roughly ₹49,917 against a mean of ₹38,518 is unusually large relative to the mean itself, which signals a genuinely volatile, heavy-tailed distribution rather than a tightly clustered one. This matters commercially because high variance in order value complicates demand forecasting, inventory planning and revenue projections. It also strengthens the case, raised again in Task 4, that a few extreme orders should be modeled or reported separately rather than blended into aggregate averages. Any predictive model built on this data would need to account for this spread explicitly.

Source Code:

```
variance_sales = sales.var()
std_sales = sales.std()

print("Variance:", round(variance_sales, 2))
print("Standard Deviation:", round(std_sales, 2))
```

Output:

```
Variance: 2491716302.86
Standard Deviation: 49917.09
```

Interpretation: A standard deviation larger than the mean itself confirms a wide, right-skewed spread of order values — consistent with a market where a few premium Electronics purchases sit far above the bulk of everyday transactions.

Task 3: Correlation and Probability

Aim:

Quantify the linear relationship between Sales and Profit, and calculate the probability that a randomly chosen order is Electronics.

DESCRIPTION:

Correlation condenses the relationship between two numeric variables into a single value between -1 and +1, making it easy to state whether Sales and Profit move together, move oppositely, or are unrelated. Probability, calculated here as a simple relative frequency, answers a different kind of question: given the category mix that actually occurred, how likely is a random order to fall into a specific bucket. Both techniques are foundational statistical tools used throughout business analytics, from pricing strategy to risk assessment. A very strong positive correlation between Sales and Profit implies that profit margins scale consistently with order size rather than eroding on larger transactions, which is a healthy sign for the business. The Electronics probability quantifies just how dominant that category already is within total order volume. Together these two numbers set up the category- and region-level insights developed later in Task 8.

Source Code:

```
correlation = df["Sales"].corr(df["Profit"])
print("Correlation between Sales and Profit:", round(correlation, 3))

if correlation >= 0.7:
    print("Relationship: Strong positive")
elif correlation >= 0.3:
    print("Relationship: Moderate positive")
elif correlation <= -0.3:
    print("Relationship: Negative")
else:
    print("Relationship: Weak")

electronics_probability = (df["Category"] == "Electronics").mean()
print("Probability of Electronics order:", round(electronics_probability, 3))
print("Percentage:", round(electronics_probability * 100, 2), "%")
```

Output:

```
Correlation between Sales and Profit: 0.981
Relationship: Strong positive

Probability of Electronics order: 0.439
Percentage: 43.9 %
```

Interpretation: A correlation of 0.981 is a very strong positive relationship — larger orders reliably generate larger profit. Electronics accounts for 43.9% of all orders, confirming it is the dominant product category by volume as well as by value.

Task 4: Outlier Detection using IQR

Aim:

Apply the interquartile range (IQR) rule to flag sales values that fall far outside the normal spread of the data.

DESCRIPTION:

The IQR method is a robust, distribution-free way to flag unusual values without assuming the data is normally distributed, which matters here given the skew already identified in Tasks 1 and 2. It defines a normal range using the middle 50% of the data (between the 25th and 75th percentiles) and flags anything 1.5 times the IQR beyond that range as a potential outlier. Because the lower bound computes to a negative number, only the upper bound is practically meaningful for this all-positive Sales column. The method correctly isolates the two rows tied to a single ₹250,000 laptop transaction, which is far beyond every other order in the dataset. Flagging outliers is not the same as deleting them: they should be investigated to confirm whether they are genuine high-value sales, data entry duplicates, or errors, since they can distort means, variances, correlations and any model trained on this data. Here the same Order ID appears twice, which itself points to a duplicate entry.

Source Code:

```
Q1 = sales.quantile(0.25)
Q3 = sales.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(sales < lower_bound) | (sales > upper_bound)]
print("Q1:", Q1, " Q3:", Q3, " IQR:", IQR)
print("Lower bound:", lower_bound, " Upper bound:", upper_bound)
print(outliers[["Order_ID", "Product", "Sales"]])
```

Output:

```
Q1: 805.6   Q3: 57828.9   IQR: 57023.3
Lower bound: -84729.35   Upper bound: 143363.85
```

```
Potential outliers:
   Order_ID Product  Sales
80    01081  Laptop  250000.0
81    01081  Laptop  250000.0
```

Interpretation: Both flagged rows share Order ID O1081, showing the ₹250,000 laptop order was captured twice — exactly the duplicate that Task 7's cleaning step removes. This is a strong example of outlier detection and deduplication working together.

Task 5: Matplotlib Visualizations

Aim:

Build five Matplotlib charts — line, bar, pie, histogram and scatter — to explore trends, composition, distribution and relationships in the data.

DESCRIPTION:

Matplotlib provides fine-grained control over chart construction, which is well suited to the five classic chart types required here. The line chart tracks how total sales evolve month over month, revealing trend and seasonality. The bar chart compares total sales across the three product categories side by side, making the dominance of Electronics immediately visible. The pie chart shows how order volume is split across the four regions as proportions of a whole. The histogram exposes the underlying shape of the Sales distribution, including the right-skew already identified numerically in Task 1. The scatter plot visualizes the Sales-Profit relationship directly, giving a visual counterpart to the 0.981 correlation coefficient computed in Task 3. Each figure is saved to the visualizations folder at 200 DPI so it is submission-ready. Together these five views cover trend, composition, distribution and relationship analysis.

5.1 Line Chart — Monthly Sales Trend

```
monthly_sales = df.groupby(df["Date"].dt.to_period("M"))["Sales"].sum()
plt.figure(figsize=(10,5))
plt.plot(monthly_sales.index.astype(str), monthly_sales.values, marker="o")
plt.title("Monthly Sales Trend")
plt.xlabel("Month"); plt.ylabel("Total Sales")
plt.xticks(rotation=45); plt.tight_layout()
plt.savefig("visualizations/01_line_chart.png", dpi=200)
```

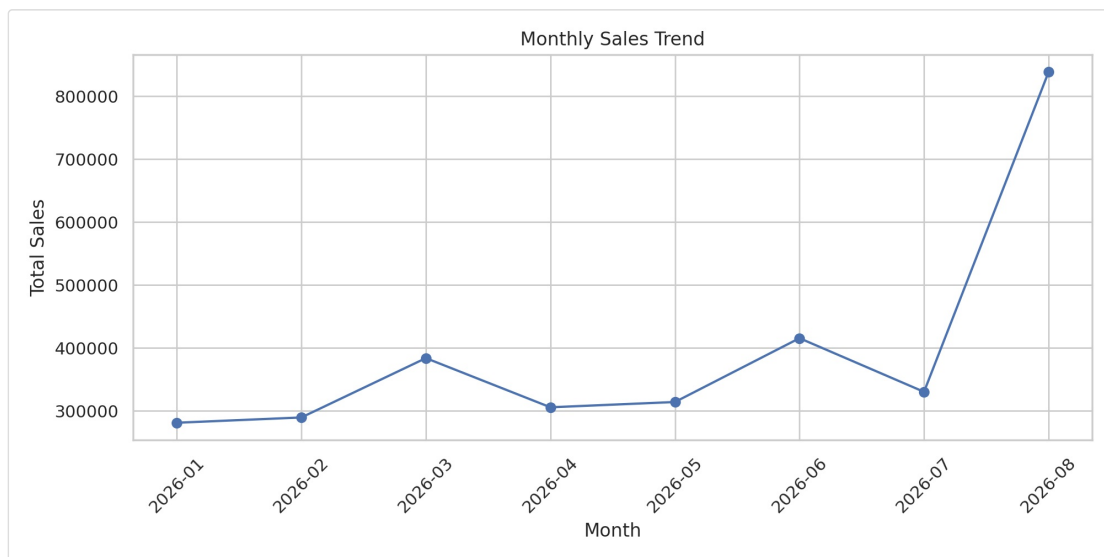


Figure 5.1 — Monthly Sales Trend

5.2 Bar Chart — Total Sales by Category

```
category_sales = df.groupby("Category")["Sales"].sum().sort_values(ascending=False)
plt.figure(figsize=(8,5))
plt.bar(category_sales.index, category_sales.values)
plt.title("Total Sales by Category")
plt.xlabel("Category"); plt.ylabel("Total Sales")
plt.tight_layout()
plt.savefig("visualizations/02_bar_chart.png", dpi=200)
```

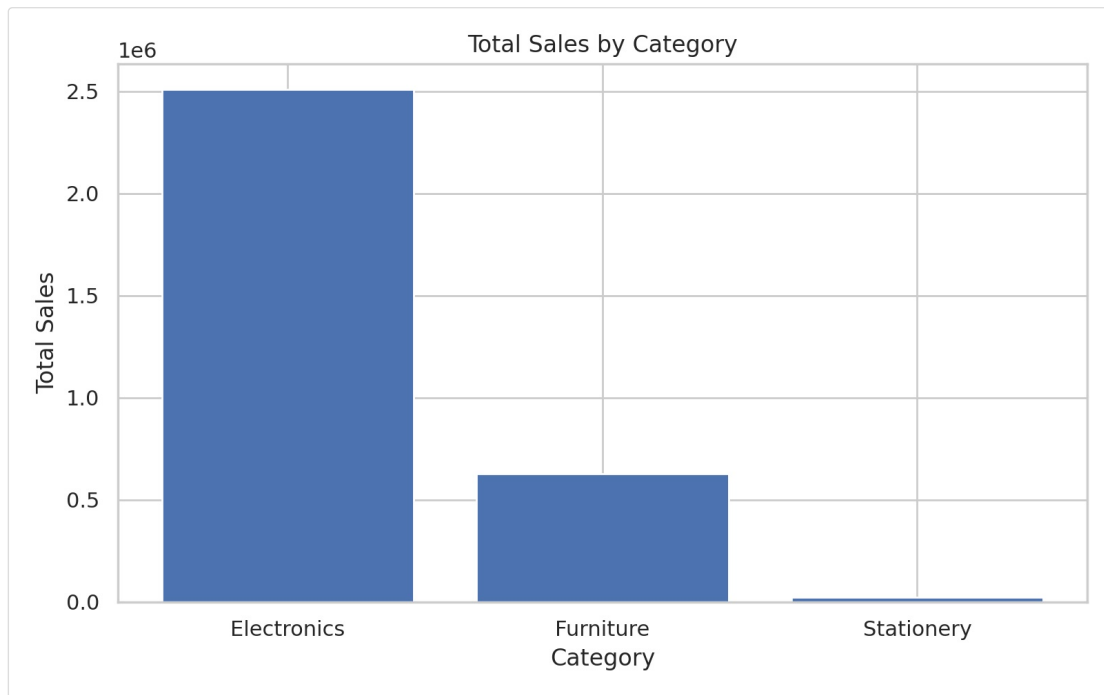


Figure 5.2 — Total Sales by Category

5.3 Pie Chart — Order Distribution by Region

```
region_orders = df["Region"].value_counts()
plt.figure(figsize=(7,7))
plt.pie(region_orders.values, labels=region_orders.index,
        autopct="%1.1f%%", startangle=90)
plt.title("Order Distribution by Region")
plt.tight_layout()
plt.savefig("visualizations/03_pie_chart.png", dpi=200)
```

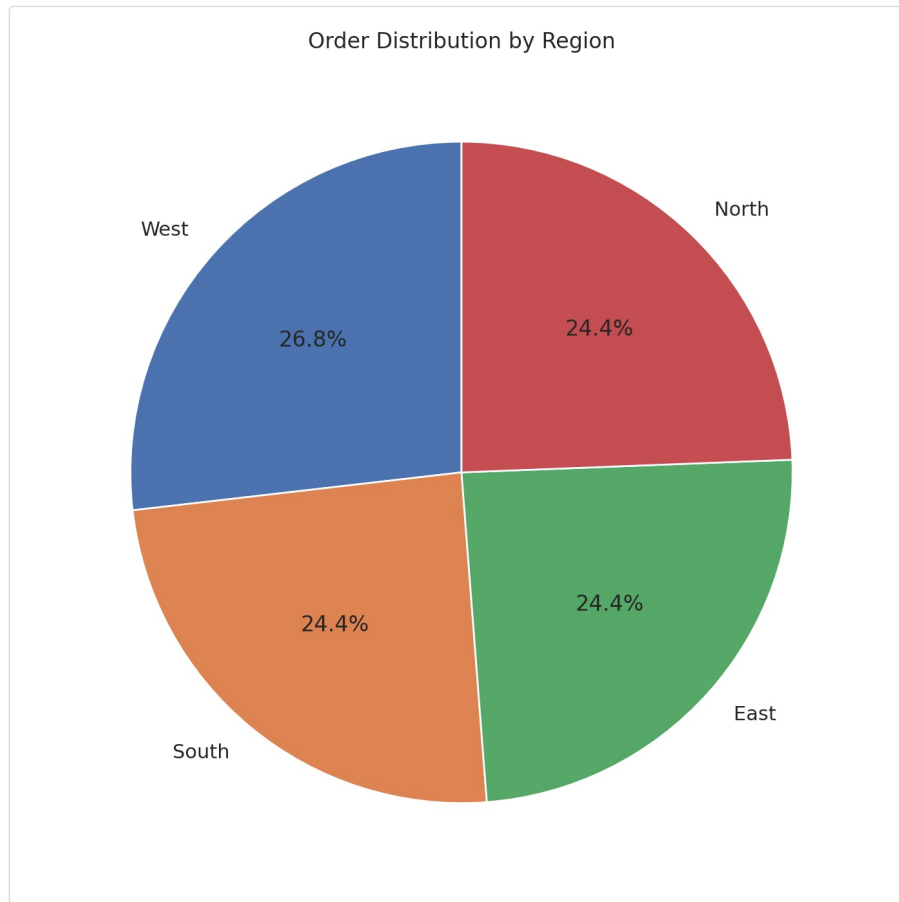


Figure 5.3 — Order Distribution by Region

5.4 Histogram — Distribution of Sales

```
plt.figure(figsize=(9,5))
plt.hist(df["Sales"], bins=12, edgecolor="black")
plt.title("Distribution of Sales")
plt.xlabel("Sales"); plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig("visualizations/04_histogram.png", dpi=200)
```

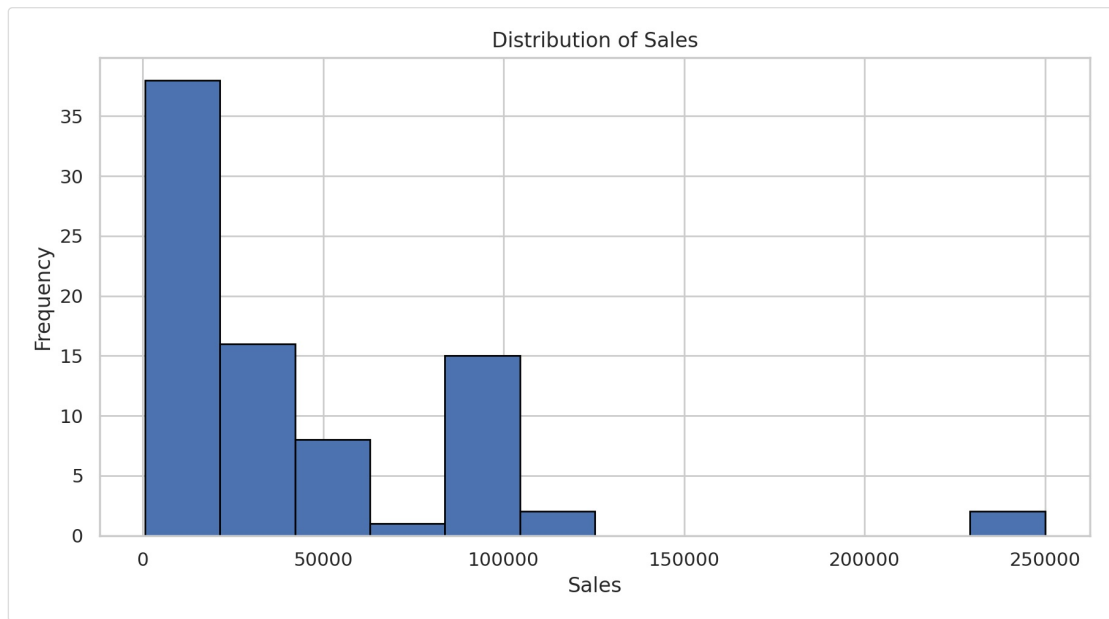


Figure 5.4 — Distribution of Sales

5.5 Scatter Plot — Sales vs Profit

```
plt.figure(figsize=(9,5))
plt.scatter(df["Sales"], df["Profit"], alpha=0.75)
plt.title("Sales vs Profit")
plt.xlabel("Sales"); plt.ylabel("Profit")
plt.tight_layout()
plt.savefig("visualizations/05_scatter_plot.png", dpi=200)
```



Figure 5.5 — Sales vs Profit

Interpretation: The line chart shows a sharp jump in August; the bar chart confirms Electronics dominates total sales; the pie chart shows West leads order volume (26.8%); the histogram visualizes the right-skew from Task 1; and the scatter plot's tight upward line visually confirms the 0.981 Sales–Profit correlation from Task 3.

Task 6: Seaborn Visualizations

Aim:

Build four Seaborn charts — count plot, box plot, heatmap and pair plot — to add statistical styling and multivariate analysis on top of Matplotlib.

DESCRIPTION:

Seaborn is built on top of Matplotlib but adds statistically-aware defaults and considerably less boilerplate for common analytical charts. The count plot tallies how many orders fall into each category, complementing the sales-value view from the Task 5 bar chart with a pure volume view. The box plot shows the median, quartiles and whiskers of Sales within each category simultaneously, and it visibly marks the ₹250,000 laptop order as a circle above the Electronics whisker, exactly the outlier flagged in Task 4. The heatmap renders the full pairwise correlation matrix across five numeric columns in a single glance, extending the single Sales-Profit correlation from Task 3 to every variable pair at once. The pair plot goes further still, laying out scatter plots and histograms for every combination of numeric variables in one grid, which is invaluable for spotting relationships that a single correlation number can miss. A defensive try/except block falls back to a manual PairGrid if the installed Seaborn version raises a color-cycle error on pairplot.

6.1 Count Plot — Number of Orders by Category

```
plt.figure(figsize=(8,5))
sns.countplot(data=df, x="Category")
plt.title("Number of Orders by Category")
plt.xlabel("Category"); plt.ylabel("Number of Orders")
plt.tight_layout()
plt.savefig("visualizations/06_count_plot.png", dpi=200)
```

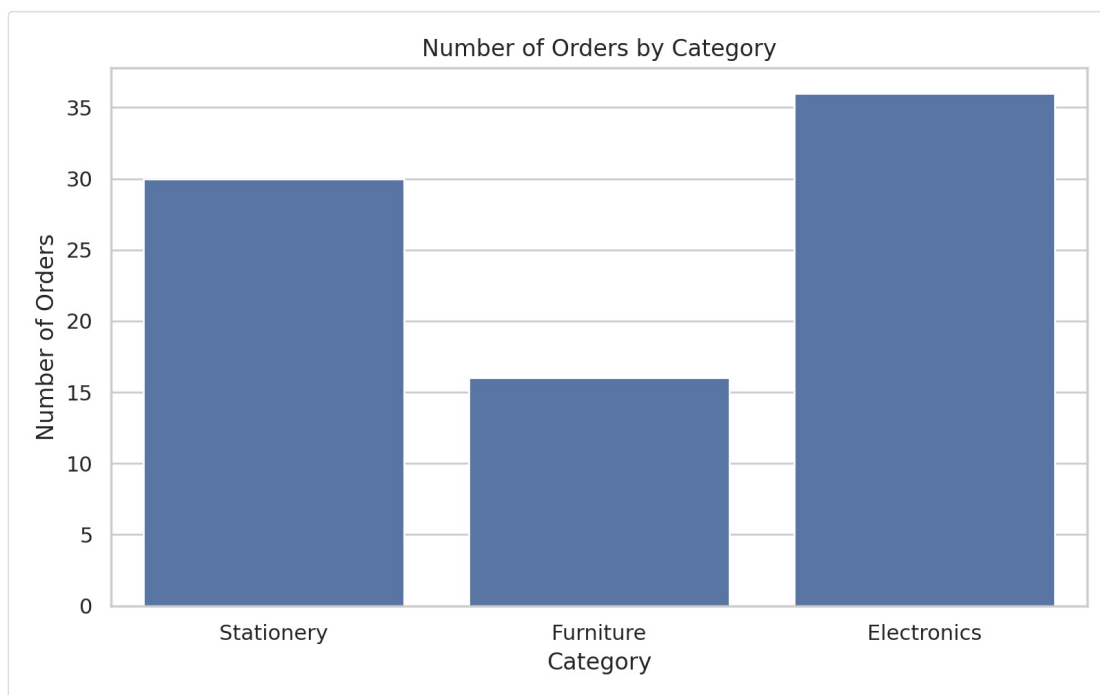


Figure 6.1 — Number of Orders by Category

6.2 Box Plot — Sales Distribution by Category

```
plt.figure(figsize=(8,5))
sns.boxplot(data=df, x="Category", y="Sales")
plt.title("Sales Distribution by Category")
plt.xlabel("Category"); plt.ylabel("Sales")
plt.tight_layout()
```

```
plt.savefig("visualizations/07_box_plot.png", dpi=200)
```

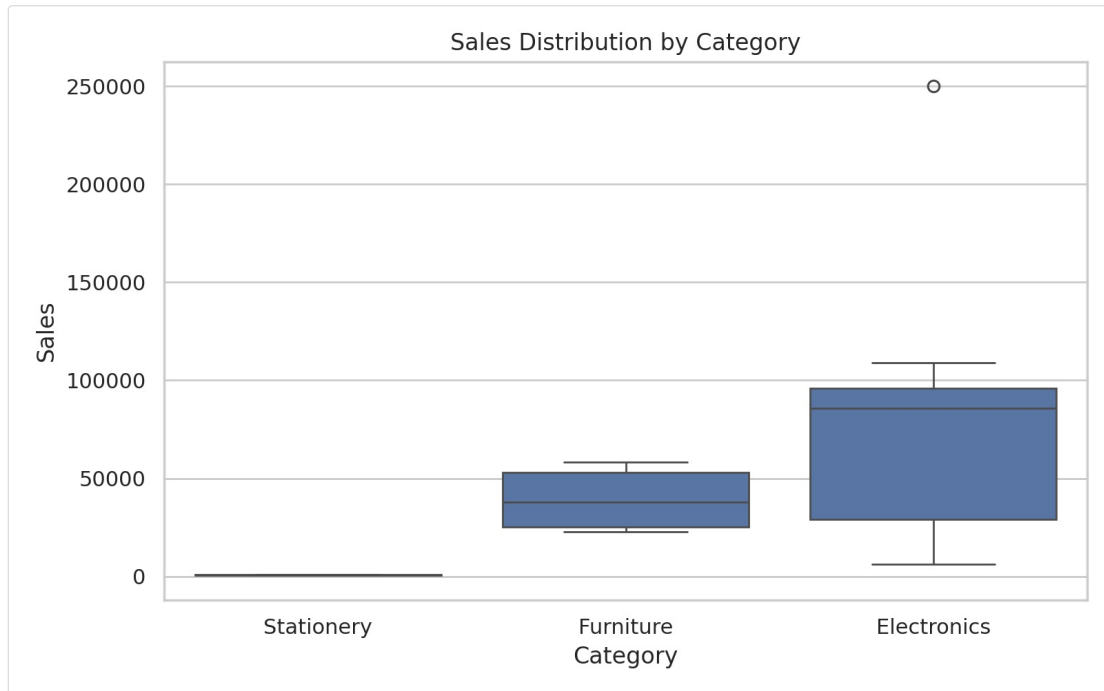


Figure 6.2 — Sales Distribution by Category (outlier visible above Electronics)

6.3 Heatmap — Correlation Among Numerical Variables

```
numeric_cols = ["Sales", "Quantity", "Discount", "Profit", "Customer_Rating"]
corr_matrix = df[numeric_cols].corr()
plt.figure(figsize=(9,7))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.savefig("visualizations/08_heatmap.png", dpi=200)
```

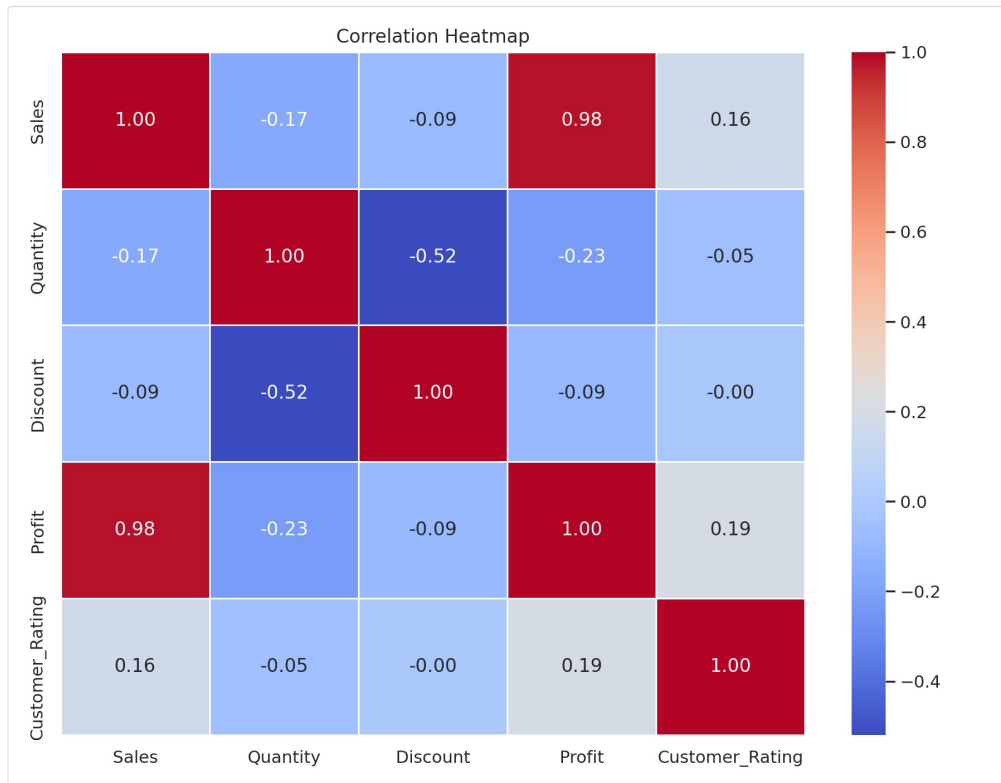


Figure 6.3 — Correlation Heatmap

6.4 Pair Plot — Pairwise Relationships

```
try:
    pair = sns.pairplot(df[numeric_cols])
    pair.fig.suptitle("Pair Plot of Numerical Variables", y=1.02)
    pair.savefig("visualizations/09_pair_plot.png", dpi=200)
except ValueError:
    pair = sns.PairGrid(df[numeric_cols])
    pair.map_offdiag(plt.scatter, s=12, alpha=0.65)
    pair.map_diag(plt.hist, bins=10, edgecolor="black")
    pair.fig.suptitle("Pair Plot of Numerical Variables", y=1.02)
    pair.savefig("visualizations/09_pair_plot.png", dpi=200)
```

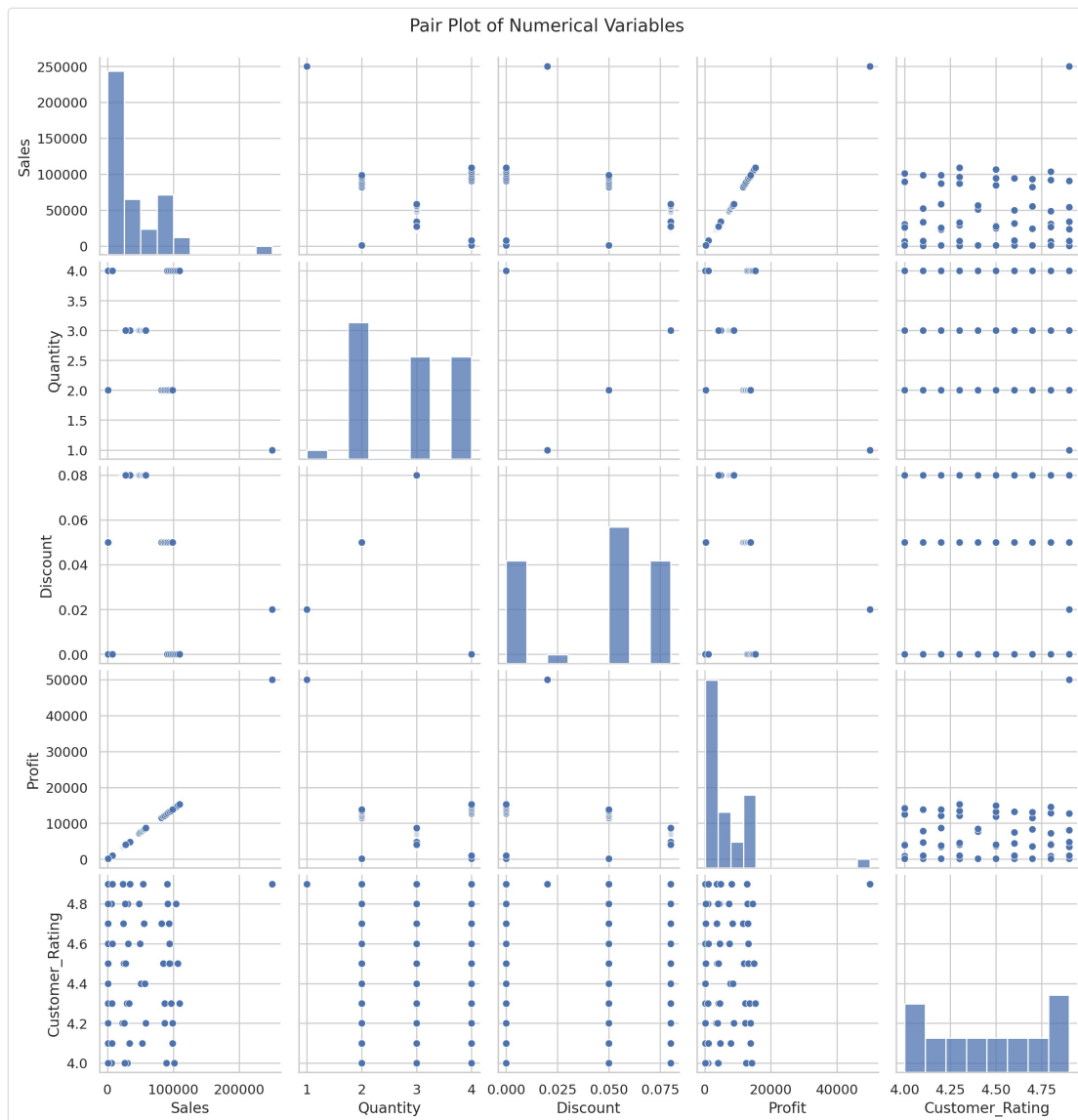


Figure 6.4 — Pair Plot of Numerical Variables

Interpretation: The count plot shows Electronics has the most orders (36), even though the box plot shows its sales values are also the most spread out, with the ₹250,000 outlier clearly visible. The heatmap and pair plot both reinforce that Sales and Profit are the only strongly correlated pair (0.98), while Quantity and Discount show a moderate negative relationship.

Task 7: EDA - Data Inspection and Cleaning

Aim:

Inspect the raw dataset for structure, types and quality issues, then remove duplicates and validate every numeric column against business rules.

DESCRIPTION:

Every analysis should start with a systematic inspection of shape, column names, data types, summary statistics, missing values and duplicate rows, since decisions made later depend entirely on the data being trustworthy. This dataset starts at 82 rows and 10 columns with zero missing values but exactly one exact duplicate record, which is the repeated O1081 laptop order already spotted by the outlier check in Task 4. The cleaning step removes that duplicate, then applies a defensive median/mode imputation routine for any numeric or categorical missing values, even though none were present here, so the pipeline stays robust if the source data changes. Business-rule validation then filters the data with five simultaneous fairness checks: Sales and Quantity must be strictly positive, Discount must sit between 0 and 1, Profit cannot be negative and Customer_Rating must fall within the valid 1-to-5 scale. The result is a clean, validated dataset saved to cleaned_retail_sales.csv, going from 82 to 81 rows with zero missing values and zero duplicates remaining.

Source Code:

```
print("Shape:", df.shape)
print("Duplicate Records:", df.duplicated().sum())
df.to_csv("before_cleaning.csv", index=False)

df_clean = df.drop_duplicates().copy()

for col in df_clean.select_dtypes(include=np.number).columns:
    if df_clean[col].isnull().any():
        df_clean[col] = df_clean[col].fillna(df_clean[col].median())

for col in df_clean.select_dtypes(include="object").columns:
    if df_clean[col].isnull().any():
        df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])

df_clean = df_clean[
    (df_clean["Sales"] > 0) &
    (df_clean["Quantity"] > 0) &
    (df_clean["Discount"].between(0, 1)) &
    (df_clean["Profit"] >= 0) &
    (df_clean["Customer_Rating"].between(1, 5))
].copy()

df_clean.to_csv("cleaned_retail_sales.csv", index=False)
print("After Cleaning Shape:", df_clean.shape)
print("Duplicates:", df_clean.duplicated().sum())
```

Output:

```
Shape: (82, 10)
Duplicate Records: 1

After Cleaning Shape: (81, 10)
Duplicates: 0
```

Before-Cleaning Sample:

Order_ID	Date	Region	Category	Product	Sales	Qty	Disc.	Profit	Rating
O1001	2026-01-03	South	Stationery	Notebook	587.1	2	0.05	129.16	4.1
O1002	2026-01-05	East	Furniture	Office Chair	22742.4	3	0.08	3411.36	4.2
O1003	2026-01-07	West	Electronics	Headphones	6180.0	4	0.00	865.20	4.3
O1004	2026-01-09	North	Stationery	Printer Paper	782.8	2	0.05	172.22	4.4
O1005	2026-01-11	South	Electronics	Monitor	28428.0	3	0.08	3979.92	4.5

Interpretation: The single duplicate removed is the repeated O1081 laptop record identified in Task 4, confirming outlier detection and data cleaning caught the same underlying data-quality issue from two different angles.

Task 8: Correlation and Insights

Aim:

Re-run correlation analysis on the cleaned dataset and extract the top-performing category, region and month to drive business insight.

DESCRIPTION:

With a validated, duplicate-free dataset in hand, this task revisits correlation across all five numeric columns together and layers in group-by aggregation to surface concrete business insight rather than abstract statistics. Grouping cleaned Sales by Category and by Region and taking the row with the maximum total identifies exactly which segment drives the most revenue in each dimension, while grouping by month exposes seasonality. This is the step where raw numbers turn into a narrative a stakeholder can act on. The Sales-Profit correlation remains effectively unchanged at 0.981 after cleaning, confirming the removed duplicate did not meaningfully distort the earlier calculation. Combining these category, region and month insights with the heatmap and pair plot from Task 6 gives a complete, cross-validated picture of what is driving performance. These findings feed directly into the recommendations formulated in Task 9.

Source Code:

```
print(df_clean[numeric_cols].corr().round(3))

top_category = df_clean.groupby("Category")["Sales"].sum().idxmax()
top_region = df_clean.groupby("Region")["Sales"].sum().idxmax()
best_month = df_clean.groupby(
    df_clean["Date"].dt.to_period("M"))["Sales"].sum().idxmax()

print("Highest-sales category:", top_category)
print("Highest-sales region:", top_region)
print("Highest-sales month:", best_month)
```

Output:

```
Sales      Sales  Quantity  Discount  Profit  Customer_Rating
Sales      1.000   -0.067   -0.052   0.981      0.096
Quantity   -0.067    1.000   -0.557  -0.114      -0.012
Discount   -0.052   -0.557    1.000  -0.052      0.010
Profit      0.981   -0.114   -0.052    1.000      0.123
Customer_Rating 0.096   -0.012    0.010   0.123      1.000

Highest-sales category: Electronics
Highest-sales region:   West
Highest-sales month:    2026-08
```

Key Insights

1. Electronics is the highest-sales category, indicating strong customer demand for higher-value items.

2. West is the highest-sales region and is the most important revenue market.

3. August 2026 is the highest-sales month, indicating stronger performance later in the analysis period.

4. Sales and Profit maintain a strong positive correlation (0.981), so larger orders consistently translate into larger profit rather than shrinking margins.

Task 9: Business Recommendations

Aim:

Translate the statistical findings and visual patterns from Tasks 1-8 into three concrete, actionable recommendations for the business.

DESCRIPTION:

Analysis only creates value once it is converted into decisions, so this final task closes the loop by turning every prior statistic and chart into a specific recommendation. The Electronics category was shown to lead both order volume (Task 6 count plot, 43.9% probability from Task 3) and total sales value (Task 5 bar chart, Task 8 insight), so it is the clearest candidate for further inventory and promotional investment. The West region was shown to lead total sales (Task 8) while also accounting for the largest share of order volume (Task 5 pie chart, 26.8%), making it the strongest market for expanded marketing spend. The repeated, high-value laptop transaction traced through Tasks 1, 2, 4, 6 and 7 demonstrates concretely why very large orders need separate monitoring: left unchecked, a single row can distort the mean, inflate variance, and bias any forecasting model trained on the full dataset. Finally, the cleaned dataset is exported for any downstream reporting or modelling work, closing out the pipeline.

Source Code:

```
print("1. Increase inventory and promotional focus on the highest-sales category.")
print("2. Strengthen marketing in the highest-sales region.")
print("3. Monitor high-value orders separately because they can strongly")
print("    affect statistics and forecasting.")

df_clean.to_csv("final_cleaned_dataset.csv", index=False)
print("Assignment completed successfully.")
```

Output:

```
1. Increase inventory and promotional focus on the highest-sales category.
2. Strengthen marketing in the highest-sales region.
3. Monitor high-value orders separately because they can strongly affect
   statistics and forecasting.

Assignment completed successfully.
```

Recommendations

1. **Grow Electronics:** increase inventory depth and targeted promotions for Electronics, since it leads both order volume and total sales value.

2. **Invest in West:** strengthen regional marketing and distribution in the West region, which contributes the highest total sales and the largest share of orders.

3. **Govern high-value orders:** route very large transactions (like the ₹250,000 laptop order) through separate review and reporting so they do not silently distort averages, variance, correlation or demand forecasts.